

Plan for Scala

- The basics
- Control structures and functions
- Collections
- Case classes
- Structure of a Scala program

Interactive mode - the Scala interpreter

- Instructions for installing Scala can be found here <http://horstmann.com/scala/install/>
- This link is related to the book I am referring in these slides, considered by Martin Odersky as the best book for quickly learning the essentials of Scala

Interactive mode - the Scala interpreter

```
scala> 8 * 5 + 2  
res0: Int = 42
```

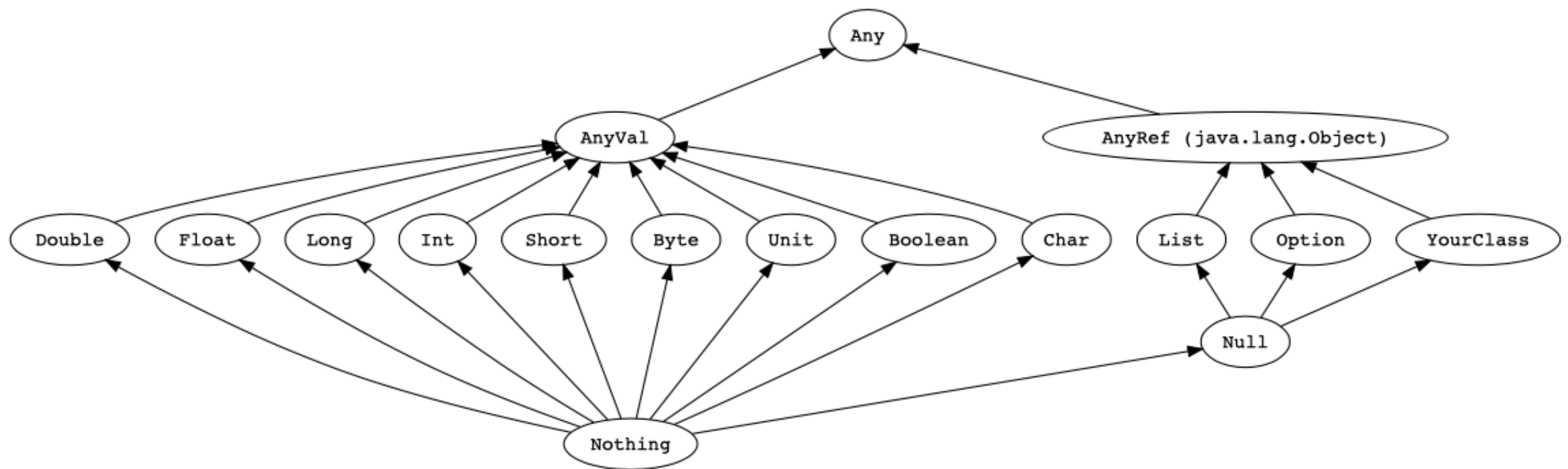
```
scala> 0.5 * res0  
res57: Double = 21.0
```

```
scala> "Hello, " + res0  
res58: String = Hello, 42
```

```
scala> 1 to 10  
res0: scala.collection.immutable.Range.Inclusive = Range 1 to 10
```

```
scala>
```

Scala types



Scala Type Hierarchy

<https://docs.scala-lang.org/tour/unified-types.html>

Focus on range collections

```
scala> 1 to 10
```

```
res1: scala.collection.immutable.Range.Inclusive = Range 1 to 10
```

```
scala> 1 until 10
```

```
res2: scala.collection.immutable.Range = Range 1 until 10
```

```
scala> 1 to 10 by 2
```

```
res3: scala.collection.immutable.Range = inexact Range 1 to 10 by 2
```

```
scala>
```

Range for populating sequences

```
scala> val x = (1 to 10).toList  
x: List[Int] = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
scala> val x = (1 to 10).toArray  
x: Array[Int] = Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
scala> val x = (1 to 10).toSet  
x: scala.collection.immutable.Set[Int] = Set(5, 10, 1, 6, 9, 2, 7, 3, 8, 4)
```

```
scala>
```

Values and Variables

```
scala> val answer = 8 * 5 + 2  
answer: Int = 42
```

```
scala> 0.5 * answer  
res59: Double = 21.0
```

```
scala> answer = 0  
<console>:13: error: reassignment to val  
      answer = 0  
             ^
```

```
scala> var counter = 0  
counter: Int = 0
```

```
scala> counter = 1  
counter: Int = 1
```

```
scala>
```

- Names can be either of the *val* or variable *kind*
- *Val* is used for names associated to constants
- While *variable* is for names whose value may change
- Suggestion: use as much as possible *val* names in programs.

Control structures and functions

- Differently from Java and C++, in Scala *almost* all constructs have a value
- For instance
 - An if-expression has a value
 - A { ; ...;;... } block has a value, the value of the last expression in the block

Conditional expressions

```
scala> val x=1  
x: Int = 1
```

```
scala> if (x > 0) 1 else -1  
res1: Int = 1
```

```
scala> val s = if (x > 0) 1 else -1  
s: Int = 1
```

```
scala>
```

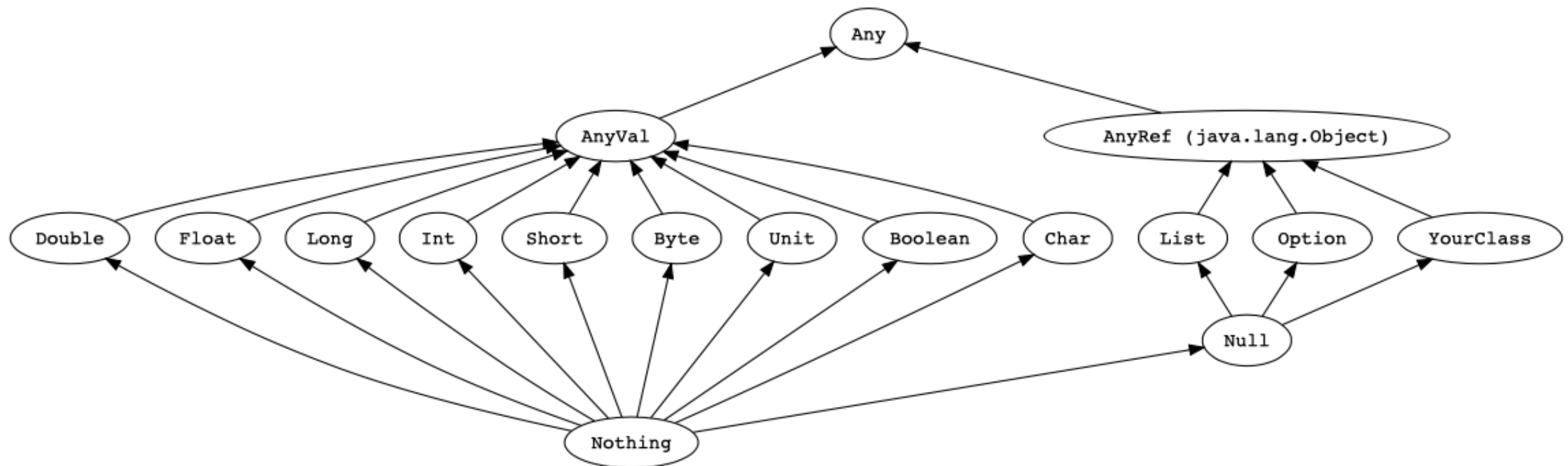
- The value of the if-expression is the value of the expression following if or else
- It follows that the if-expression has a type, the type of its value
- What happens if the if-expression may yield values of different types ?

Conditional expressions

```
scala> if (x > 0) "positive" else -1  
res2: Any = positive
```

```
scala>
```

- The type of the if-expression is Any, which is the *common* super-type of all types



Scala Type Hierarchy

Conditional expressions

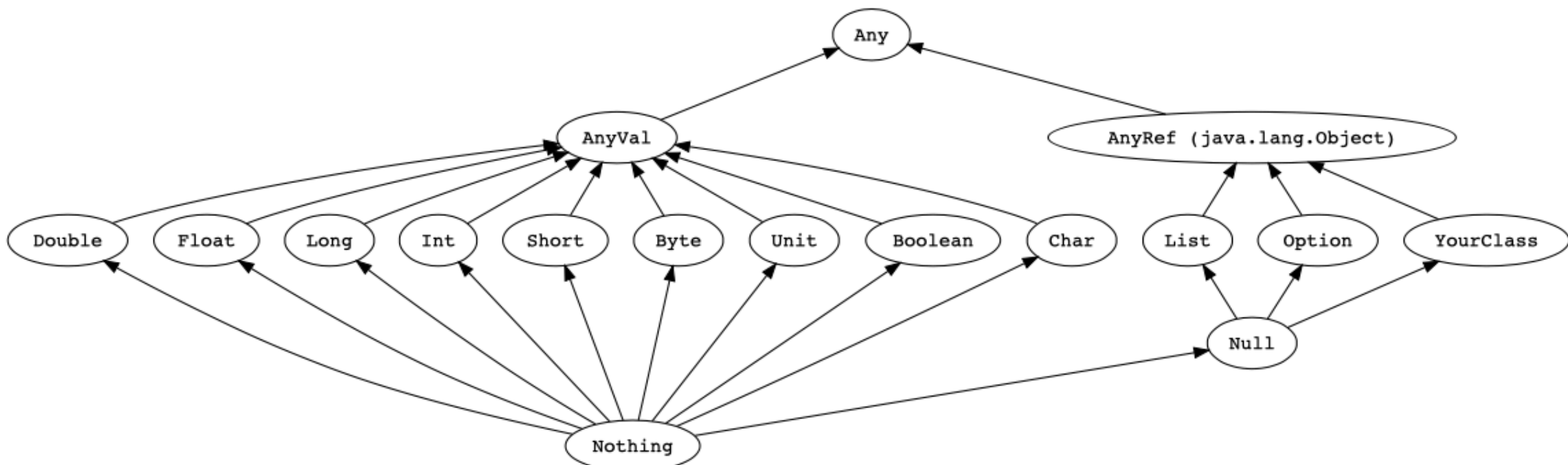
```
scala> var y=0  
y: Int = 0
```

```
scala> if (y > 0) 1  
res4: AnyVal = ()
```

```
scala> y=1  
y: Int = 1
```

```
scala> if (y > 0) 1  
res5: AnyVal = 1
```

- An if-expression without the else-branch can yield no value
- In this case Scala assume that a value is returned which is ()
- () denotes 'no value' and its type is Unit.



Complex branches

```
scala> if (n > 0) { r = r * n; n -= 1 }
```

```
scala> if (n > 0) {  
  |   r = r * n  
  |   n -= 1  
  | }
```

```
scala>
```

- Many Scala programmers prefer the second style

Block expressions and assignments

- The value/type of a block expression is the value/type of its last expression

```
scala> var x0,y0=0.1  
x0: Double = 0.1  
y0: Double = 0.1
```

```
scala> { val dx = x - x0; val dy = y - y0; sqrt(dx * dx + dy * dy) }  
res40: Double = 1.2727922061357855
```

```
scala>
```

Loops

..... n and r are initialized ...

```
scala> while (n > 0) { r=r* n
|   n -= 1
| }
```

```
scala> for (i <- 1 to n)
|   r=r* i
```

```
scala> val s = "Hello"
s: String = Hello
```

```
scala> var sum = 0
sum: Int = 0
```

```
scala> for (i <- 0 until s.length) // Last value for i is s.length - 1
|   sum += s(i)
```

```
scala>
```

- In case of for the variable does not need to be initialized, differently from the while case
- For other advanced uses see the documentation

<https://docs.scala-lang.org/tour/for-comprehensions.html>

Functions

```
scala> def abs(x: Double) = if (x >= 0) x else -x
```

```
abs: (x: Double)Double
```

```
scala> def fac(n : Int) = {
```

```
  | var r = 1
  | for (i <- 1 to n) r = r * i
  | r
  | }
```

```
fac: (n: Int)Int
```

```
scala>
```

```
scala> def fac(n: Int):Int =
```

```
  | if (n <= 0) 1 else n * fac(n - 1)
```

```
fac: (n: Int)Int
```

```
scala>
```

- Scala has functions in addition to methods.
- A method operates on an object, but a function doesn't
- While types of input parameters must be declared, the output type could not
- With the exception of recursive functions

Arrays

```
scala> val nums = new Array[Int](10)
nums: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

```
scala> val a = new Array[String](10)
a: Array[String] = Array(null, null, null, null, null, null, null, null, null, null)
```

```
scala> val s = Array("Hello", "World")
s: Array[String] = Array>Hello, World)
```

```
scala> s(0) = "Goodbye"
```

```
scala> s
res15: Array[String] = Array(Goodbye, World)
```

```
scala>
```

- They can be either of fixed or variable length
- These examples relate to fixed-length arrays
- Arrays are mutable, value at position i can be changed

Variable-Length Arrays

```
scala> import scala.collection.mutable.ArrayBuffer
```

```
scala> val b = ArrayBuffer[Int]()
```

```
b: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer()
```

```
scala> b += 1
```

```
res17: b.type = ArrayBuffer(1)
```

```
scala> b += (1, 2, 3, 5)
```

```
res18: b.type = ArrayBuffer(1, 1, 2, 3, 5)
```

```
scala> b ++= Array(8, 13, 21)
```

```
res19: b.type = ArrayBuffer(1, 1, 2, 3, 5, 8, 13, 21)
```

```
scala> b.trimEnd(5)
```

```
scala> b
```

```
res27: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer(1, 1, 2)
```

```
scala>
```

Traversing Arrays

```
scala> b += Array(8, 13, 21)
res19: b.type = ArrayBuffer(1, 1, 2, 3, 5, 8, 13, 21)
```

```
scala> b.trimEnd(5)
```

```
scala> for (i <- 0 until b.length)
  | println(i + ": " + b(i))
```

```
0: 1
1: 1
2: 2
```

```
scala> for (elem <- b)
  | println(elem)
```

```
1
1
2
```

```
scala>
```

Transforming Arrays

```
scala> b += Array(8, 13, 21)
```

```
res19: b.type = ArrayBuffer(1, 1, 2, 3, 5, 8, 13, 21)
```

```
scala> b.trimEnd(5)
```

```
scala> for (elem <- b if elem % 2 == 0) yield 2 * elem
```

```
res25: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer(4)
```

```
scala> b.filter(x => x % 2 == 0 ).map(x => 2 * x)
```

```
res26: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer(4)
```

```
scala> b.filter(_ % 2 == 0).map(2 * _)
```

```
res27: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer(4)
```

```
scala>
```

Tuples - immutable

```
scala> val t = (1, 3.14, "Fred")  
t: (Int, Double, String) = (1, 3.14, Fred)
```

```
scala> t._1  
res49: Int = 1
```

```
scala> val (first, second, third) = t  
first: Int = 1  
second: Double = 3.14  
third: String = Fred
```

```
scala> val (first, second, _) = t  
first: Int = 1  
second: Double = 3.14
```

pattern matching on tuples

Case classes and pattern matching

```
scala> case class Message(sender: String, recipient: String, body: String)
```

```
defined class Message
```

```
scala> val message1 = Message("guillaume@quebec.ca", "jorge@catalonia.es", "Ça va ?")
```

```
message1: Message = Message(guillaume@quebec.ca,jorge@catalonia.es,Ça va ?)
```

```
scala> println(message1.sender)
```

```
guillaume@quebec.ca
```

```
scala>
```

Pattern matching on case classes

```
abstract class Notification
```

```
case class Email(sender: String, title: String, body: String) extends Notification
```

```
case class SMS(caller: String, message: String) extends Notification
```

```
case class VoiceRecording(contactName: String, link: String) extends Notification
```

Pattern matching on case classes

```
abstract class Notification
```

```
case class Email(sender: String, title: String, body: String) extends Notification
```

```
case class SMS(caller: String, message: String) extends Notification
```

```
case class VoiceRecording(contactName: String, link: String) extends Notification
```

```
def showNotification(notification: Notification): String = {  
  notification match {  
    case Email(sender, title, _) =>  
      s"You got an email from $sender with title: $title"  
    case SMS(number, message) =>  
      s"You got an SMS from $number! Message: $message"  
    case VoiceRecording(name, link) =>  
      s"You received a Voice Recording from $name! Click the link to hear it: $link"  
  }  
}  
  
val someSms = SMS("12345", "Are you there?")  
val someVoiceRecording = VoiceRecording("Tom", "voicerecording.org/id/123")
```

```
println(showNotification(someSms)) // prints You got an SMS from 12345! Message: Are you there?
```

```
println(showNotification(someVoiceRecording)) // prints You received a Voice Recording from Tom! Click the link to hear it:  
voicerecording.org/id/123
```

What to use in place of numpy?

- You can rely on Nd4J
- Used in this nice post presenting a Scala implementation of linear regression

<https://www.cpuheater.com/scala/machine-learning-scala-linear-regression/>

Much more in the following references

- Scala for the impatient. *Cay Horstmann*
- Programming in Scala, 3rd Edition. *Martin Odersky et al.*
- Scala Cook book, Recipes for Object-Oriented and Functional Programming By *Alvin Alexander*